SR UNIVERSITY

School of Computer Science and Artificial Intelligence

Lab Assignment# 16.2

Program : B. Tech (CSE)
Specialization
Course Title : AI Assisted Coding
Course Code : 23CS002PC304
Semester : II
Academic Session : 2025-2026
Name of Student: A.vamshikrishna
Enrollment No. :2403A51L47
Batch No :52
Date : 17/03/26

Task 1: (Design Database Schema for Student Management System)

Input:

Schema SQL

```
CREATE TABLE Students (
    student_id INT PRIMARY KEY,
    name VARCHAR(100),
    email VARCHAR(100)
);

CREATE TABLE Courses (
    course_id INT PRIMARY KEY,
    course_name VARCHAR(100),
    credits INT
);

CREATE TABLE Enrollments (
    enrollment_id INT PRIMARY KEY,
    student_id INT,
    course_id INT,
    FOREIGN KEY (student_id) REFERENCES Students(student_id),
    FOREIGN KEY (course_id) REFERENCES Courses(course_id)
);

-- SHOW OUTPUT
SHOW TABLES;
DESCRIBE Students;
DESCRIBE Courses;
DESCRIBE Enrollments;
```

Output:

Results

Copy as Markdown

Query #5 Execution time: 0.33ms

Field	Type	Null	Key	Default	Extra
student_id	int(11)	NO	PRI	null	
name	varchar(100)	YES		null	
email	varchar(100)	YES		null	

Query #6 Execution time: 0.32ms

Field	Type	Null	Key	Default	Extra
course_id	int(11)	NO	PRI	null	
course_name	varchar(100)	YES		null	
credits	int(11)	YES		null	

Query #7 Execution time: 0.37ms

Field	Type	Null	Key	Default	Extra
enrollment_id	int(11)	NO	PRI	null	
student_id	int(11)	YES	MUL	null	
course_id	int(11)	YES	MUL	null	

Explanation:

1. Defines structure of database using tables: Students, Courses, Enrollments.
2. Primary keys ensure unique identification of each record.
3. Foreign keys maintain referential integrity between related tables.
4. Follows normalization (INF, 2NF, 3NF) to remove redundancy and ensure consistency.

SR UNIVERSITY

Task 2: (Insert Sample Records)

Input:

Database: MySQL v5.7 ▼ ▷ Run Update Fork Load Example ☆ Star PRO

Query SQL PRO Have any feedback?

```sql
INSERT INTO Students VALUES
(1, 'Arjun', 'arjun@gmail.com'),
(2, 'Meera', 'meera@gmail.com'),
(3, 'Rahul', 'rahul@gmail.com');

INSERT INTO Courses VALUES
(101, 'Database Systems', 4),
(102, 'Data Structures', 3),
(103, 'Operating Systems', 4),
(104, 'AI Fundamentals', 3);

INSERT INTO Enrollments VALUES
(1, 1, 101),
(2, 1, 102),
(3, 2, 103),
(4, 3, 101),
(5, 3, 104);

-- ✅ SHOW RESULT
SELECT * FROM Students;
SELECT * FROM Courses;
SELECT * FROM Enrollments;
```

Output:

Results Copy as Markdown

student_id	name	email
1	Arjun	arjun@gmail.com
2	Meera	meera@gmail.com
3	Rahul	rahul@gmail.com

Query #5 Execution time: 0.07ms

course_id	course_name	credits
101	Database Systems	4
102	Data Structures	3
103	Operating Systems	4
104	AI Fundamentals	3

Query #6 Execution time: 0.06ms

enrollment_id	student_id	course_id
1	1	101
2	1	102
3	2	103
4	3	101
5	3	104

Explanation:

1. Populates tables with initial data using INSERT statements.
2. Ensures relational links by matching student_id and course_id.
3. Helps in testing database functionality with sample dataset.
4. SELECT queries verify whether data insertion is successful.

Task 3: CRUD Operations

Input:

Output:

Explanation:

1. SELECT retrieves data from tables for viewing records.
2. UPDATE modifies existing data like student email.
3. DELETE removes specific records such as enrollments.
4. Validates operations using SELECT to ensure correctness.

Task 4: Aggregate Queries

Input:

Database: MySQL v5.7 ▼ ▷ Run Update Fork Load Example Star PRO

Query SQL •Im:I

Query successfully executed in 50.44ms x

```
-- Count students per course
SELECT c.course_name, COUNT(e.student_id) AS student_count
FROM Courses c
JOIN Enrollments e ON c.course_id = e.course_id
GROUP BY c.course_name;

-- Courses with more than 1 student
SELECT c.course_name
FROM Courses c
JOIN Enrollments e ON c.course id    e.course_id
GROUP BY c.course_name
HAVING COUNT(e.student_id) > 1;
```

Output:

Database: MySQL v5.7 ▼ Update Load Example Star PRO

Results Eil

Copy as Markdown

Query #1 Execution time: 0.2ms

course_name	student_count

There are no results to be displayed.

Query #2 Execution time: 0.16ms

course_name

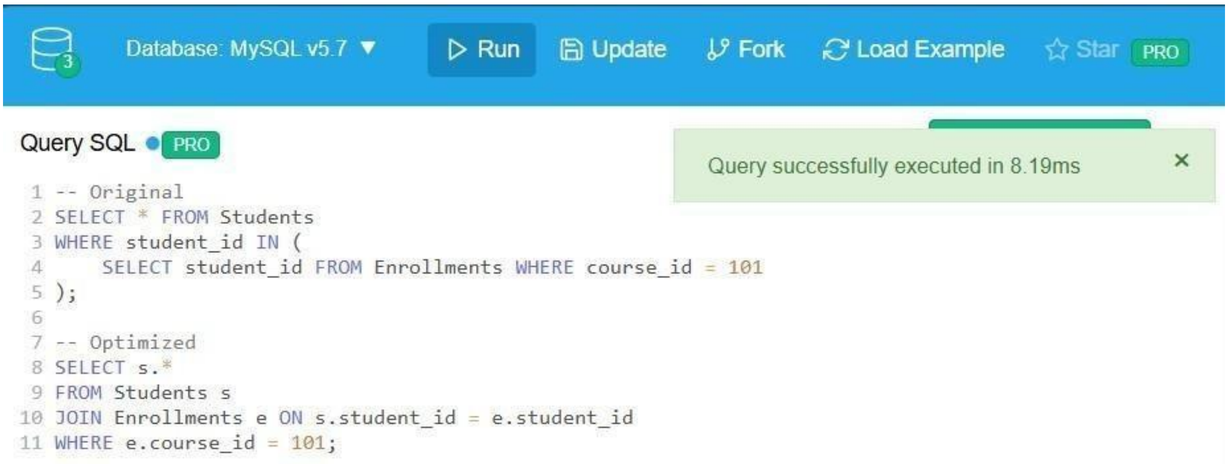
There are no results to be displayed.

Explanation:

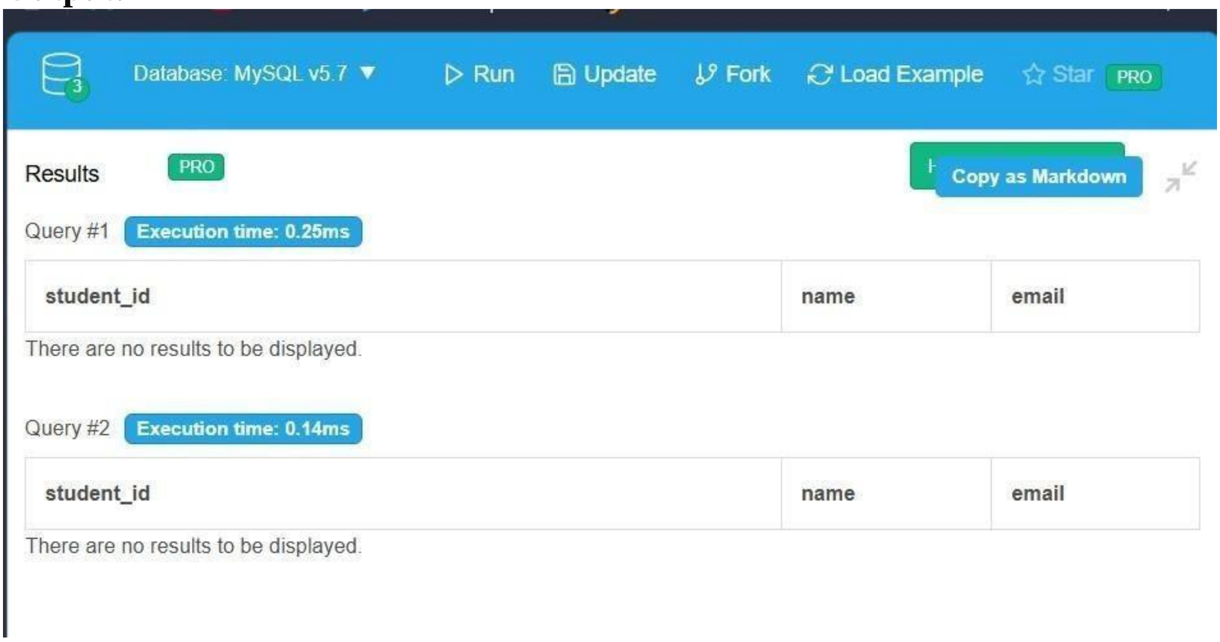
1. Uses functions like COUNT() to summarize data.
2. GROUP BY groups records based on course names.
3. HAYING filters grouped data (e.g., courses with>1 student).
4. Helps in analysis like enrollment trends per course.

Task 5: Query Optimization

Input:



Output:



Explanation:

1. Identifies inefficient queries (e.g., subqueries).
2. Rewrites queries using JOINs for better performance.
3. Ensures both original and optimized queries give same result.
4. Improves execution speed, especially for large datasets.